

ARBORI

1. **Arbore (general/liber)** = *un graf conex și aciclic*

2. **Vizualizare arbori:** https://csacademy.com/app/graph_editor/

3. Teorema de caracterizare a arborilor

Următoarele afirmații sunt echivalente pentru un graf G cu n noduri:

(1) G este un arbore.

(2) G este un graf aciclic cu $n - 1$ muchii.

(3) G este un graf conex cu $n - 1$ muchii.

(4) G este un graf aciclic maximal (dacă în graful aciclic G unim două noduri oarecare neadiacente printr-o muchie, graful obținut conține cel puțin un ciclu).

(5) G este un graf conex minimal (dacă în graful conex G suprimăm o muchie oarecare, graful obținut nu mai este conex).

(6) Orice pereche de noduri este legată printr-un lanț și numai unul.

4. Proprietăți ale arborilor:

- Orice arbore cu n noduri are $n - 1$ muchii.
- Orice arbore cu $n \geq 2$ noduri conține cel puțin două noduri terminale.

5. **Arbore cu rădăcină** = un arbore în care un nod este desemnat ca rădăcină (nivelul 0), iar restul nodurilor sunt distribuite pe niveluri (nivelurile 1, 2, ...) în raport de lungimile lanțurilor prin care sunt legate de rădăcină

Arbore general	Arbore cu rădăcina 5 și înălțimea 4	Arbore cu rădăcina 4 și înălțimea 3

Observație:

Dacă alegem ca rădăcină un vârf din centrul arborelui general, atunci înălțimea arborelui cu rădăcină va fi minimă, iar dacă alegem rădăcina din periferie, înălțimea va fi maximă!

6. **Arbore binar** = un arbore cu rădăcină în care fiecare nod are cel mult 2 descendenți / fii care se diferențiază prin poziția lor (fiu stâng / fiu drept)

7. **Vizualizare arbori binari:** <https://tree-visualizer.netlify.app/>

8. Proprietăți ale arborilor binari:

- Fiecare nivel k dintr-un arbore binar conține cel mult 2^k noduri.
- Un arbore binar de înălțime h conține cel mult $2^{h+1} - 1$ noduri.
- Într-un arbore cu n noduri și înălțime h avem relația:

$$h \geq \log_2(n + 1) - 1$$

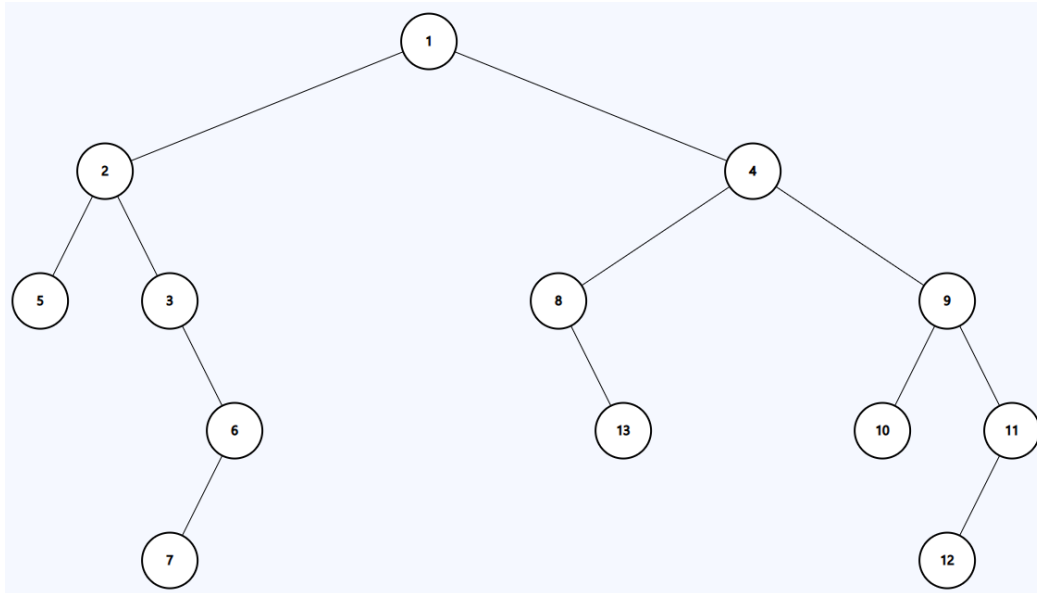
9. Tipuri de arbori binari

- *arbore binar strict* = orice nod are 0 sau 2 fii
- *arbore binar perfect* = orice nod intern are exact 2 fii și toate frunzele sunt pe același nivel
- *arbore binar complet* = toate nivelurile sunt umplute complet, posibil mai puțin ultimul nivel unde nodurile sunt completate de la stânga spre dreapta

- *arbore binar echilibrat* = pentru orice nod, valoarea absolută a diferenței dintre înălțimea subarborelui stâng și înălțimea subarborelui drept este cel mult egală cu 1

10. Metode de reprezentare ale arborilor binari

a) *vectori de fii*



Rădăcină = 1 (valoarea care nu apare ca fiu al niciunui nod)

Nod	1	2	3	4	5	6	7	8	9	10	11	12	13
Fiu stâng	2	5	0	8	0	7	0	0	10	0	12	0	0
Fiu drept	4	3	6	9	0	0	0	13	11	0	0	0	0

b) *vectori de tați + tip fiu* (-1 = fiu stânga și +1 = fiu dreapta)

Rădăcină = 1 (valoarea care nu apare ca fiu al niciunui nod)

Nod	1	2	3	4	5	6	7	8	9	10	11	12	13
Tată	0	1	2	1	2	3	6	4	4	9	9	11	8
Tip fiu	0	-1	+1	+1	-1	+1	-1	-1	+1	-1	+1	-1	+1

c) *prin rădăcină și adresele/referințele celor 2 fii* (pentru implementări într-un limbaj de programare)

11. Metode de parcurgere ale arborilor binari

Preordine (R-S-D): 1, 2, 5, 3, 6, 7, 4, 8, 13, 9, 10, 11, 12

Inordine (S-R-D): 5, 2, 3, 7, 6, 1, 8, 13, 4, 10, 9, 12, 11

Postordine (S-D-R): 5, 7, 6, 3, 2, 13, 8, 10, 12, 11, 9, 4, 1

Exemplele date se referă la arborele binar de mai sus!

12. Arbore binar de căutare (BST = Binary Search Tree) = un arbore binar în care toate nodurile respectă relația:

$$\text{valoare fiu stâng} \leq \text{valoare nod} \leq \text{valoare fiu drept}$$

13. Vizualizare arbore binar de căutare:

- <https://web.archive.org/web/20240518171540/http://btv.melezinek.cz/binary-search-tree.html>
- <https://www.cs.usfca.edu/~galles/visualization/BST.html>

14. Observații:

- Operațiile de inserare, căutare sau ștergere a unui singur nod într-un arbore binar de căutare (BST) cu n noduri au complexitatea $\mathcal{O}(\log_2 n)$ dacă BST-ul nu este degenerat, respectiv $\mathcal{O}(n)$ dacă acesta este degenerat (operațiile sunt asemănătoare cu o căutare binară).
- Parcurgerea în inordine a unui BST cu n noduri are complexitatea $\mathcal{O}(n)$.
- Parcurgerea în inordine (S-R-D) a unui BST cu n noduri va furniza valorile din nodurile sale în ordine crescătoare, iar complexitatea va fi $\mathcal{O}(n \log_2 n)$!

15. Implementare BST în Python

16. Link-uri utile:

- [Python Binary Search Tree: Find the kth smallest element in a given a binary search tree\(BST\) - w3resource](#)
- [Arbori binari de căutare | www.pbinfo.ro](http://www.pbinfo.ro)
- [Binary Search Tree - GeeksforGeeks](#)
- [Microsoft Word - arbori.docx \(mateinfo.net\)](#)
- [Arbori Binari.pdf \(panseluta.ro\)](#)

17. Arbore parțial de cost minim (Algoritmul lui Kruskal – 1956)

- [Algoritmul lui Kruskal | www.pbinfo.ro](http://www.pbinfo.ro)
- [Algoritmul lui Kruskal - Wikipedia](https://en.wikipedia.org/wiki/Kruskal's_algorithm)
- [Introduction to Disjoint Set Data Structure or Union-Find Algorithm - GeeksforGeeks](https://www.geeksforgeeks.org/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/)
- [Union By Rank and Path Compression in Union-Find Algorithm - GeeksforGeeks](https://www.geeksforgeeks.org/union-by-rank-and-path-compression-in-union-find-algorithm/)
- [Kruskal's Minimum Spanning Tree Algorithm | Greedy Algo-2 - GeeksforGeeks](https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/)